

PySpark e Apache Kafka para
processamento de dados em lote e
streaming

64%

Trilha

Fórum

Projeto 5 - Acelerando Entradas, Processamento e Armazenamento de Dados em Tempo Real - Parte 3/3

video

06:14

Projeto 5 - Convertendo e Visualizando o Conteúdo de Arquivos Parquet - Parte 1/2

videolivro

01:23

Projeto 5 - Convertendo e Visualizando o Conteúdo de Arquivos Parquet - Parte 2/2

video

03:52

Projeto 5 - Conclusão

video

02:16

Manipulação de Erros e Novas Tentativas em Cluster Kafka e Spark

e-book

01:51

Otimizações Para Alto Throughput em Cluster Kafka e Spark

e-book

01:41

Arquivos Usados Neste Capítulo

pdf

Bibliografia, Referências e Links Úteis

pdf



Otimizações Para Alto Throughput
em Cluster Kafka e Spark

Para otimizar o throughput em clusters Kafka, é essencial ajustar configurações que favoreçam o processamento em lote e o uso eficiente dos recursos.

Do lado do produtor, aumentar o tamanho dos lotes de mensagens (batch.size) e configurar um tempo de espera para acumular esses lotes (linger.ms) reduz a sobrecarga de envio.

A análise de dados, como snappy ou lz4, também diminuiu o volume transmitido e melhorou a utilização da largura de banda. O número de partições deve ser configurado para permitir maior paralelismo, mas balanceado com os recursos disponíveis no cluster.

Além disso, ajustar o tamanho do buffer no produtor (buffer.memory) e no consumidor (fetch.max.bytes) ajuda a lidar com fluxos de dados mais volumosos, enquanto reduzir as confirmações de mensagens (acks=1 ou acks=0) pode aumentar a velocidade, se algum nível de perda for aceitável.

No Spark, o throughput pode ser melhorado otimizando o particionamento de RDDs ou DataFrames para equilibrar o trabalho entre os executores. No modo de streaming, configure intervalos maiores para microlotes ou use o modo contínuo para reduzir a latência e melhorar o uso dos recursos.

O cache de dados é frequentemente reutilizado e minimiza as recomputações desnecessárias, enquanto o checkpointing garante a recuperação eficiente em caso de falhas. Além disso, a alocação de memória e núcleos para os executores (spark.executor.memory e spark.executor.cores) evita gargalos no processamento.

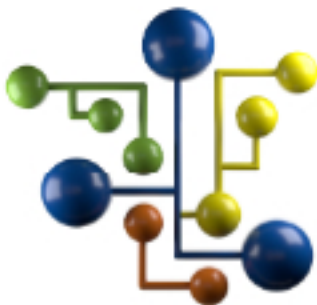
Na integração Kafka-Spark, o alinhamento entre o particionamento do Kafka e o paralelismo do Spark é fundamental para evitar sobrecarga ou subutilização de recursos. Configurar o tamanho das mensagens lidas pelo Spark (maxOffsetsPerTrigger) ajuda a equilibrar a taxa de transferência e a latência. O consumo eficiente em lotes e a configuração cuidadosa de compensações garantem que o fluxo de dados seja processado de forma rápida e confiável, maximizando o desempenho do sistema como um todo.

?

Suporte

?

Suporte



Equipe DSA

Continue trilhando uma excelente
jornada de aprendizagem.

?

Suporte

?

Suporte



?

Suporte

?

Suporte

